

```
scala> spark.sql("select * from employees").show
+-----+-----+
|  eid |  name |
+-----+-----+
|1234|Krishna|
|2345|  Mohan|
|3456|  Koyya|
+-----+-----+

scala> spark.sql("select * from employees where eid > 2000").show
+-----+-----+
|  eid |  name |
+-----+-----+
|2345|Mohana|
|3456|Koyya|
+-----+-----+
```

Figure 4: Spark SQL

projections that we normally expect in an SQL environment.

Here are some examples that are self-explanatory:

```
spark.sql("select * from employees").
show
spark.sql("select * from employees
where eid > 2000").show
```

Stream processing

A few years ago, organisations used to process data primarily in batches. For instance, a news publisher would update its online portal once an hour. However, with the penetration of the Internet and mobile devices, the demand for real-time news is only growing. Publishers are expected to update the portal as and when an event is reported. In other words, organisations are expected to process streams of data, round-the-clock.

Apache Spark has excellent support for stream processing. Though the abstractions like RDDs and DataFrames are best suited for batch processing, Spark also uses them to build scalable, fault-tolerant stream processing capabilities.

The secret of Spark's stream processing is in the way it divides the streamed data. By applying a time window, a stream of data can be treated as several batches of data. When the size of the window is

```
nc -lk 1234
Krishna
Krishna Mohan
Krishna Mohan Koyya
```

Figure 5: TCP socket data stream

```
-----
Batch: 0
-----
+-----+-----+
| value | count |
+-----+-----+

-----
Batch: 1
-----
+-----+-----+
| value | count |
+-----+-----+
| Krishna | 1 |
+-----+-----+

-----
Batch: 2
-----
+-----+-----+
| value | count |
+-----+-----+
| Koyya | 1 |
| Mohan | 2 |
| Krishna | 3 |
+-----+-----+
```

Figure 6: Spark stream processing

sufficiently small, stream processing turns out to be just micro batch processing, but at a high frequency. Spark does exactly this. In other words, a discrete stream is treated just as a sequence of RDDs.

A typical stream processing job written in Scala looks like the following:

```
val lines = spark.readStream.
format("socket").option("host",
"localhost").option("port", 1234).
load()
val words = lines.as[String].flatMap(_
.split(" "))
val counts = words.groupBy("value").
count()
val ctx = counts.writeStream.
outputMode("complete").
format("console").start
ctx.awaitTermination()
```

The above code creates a Spark job to listen to TCP socket at port 1234 on the local machine, for a stream of data, and print the word count on the console. To run the above job, a server should be started on that port, beforehand. A simple tool like netcat can be used for experimentation as follows:

```
$nc -lk 1234
```

Enter some lines of text in the server window, as shown in Figure 5. These lines will be received by the Spark job that is listening to this port, as a stream. The results are demonstrated in Figure 6.

Spark supports data sources including Apache Kafka, Apache Flume, AWS Kinesis, TCP sockets, files, etc, out-of-the-box. When it comes to processing, it supports algorithms including but not limited to map-reduce, join, and sliding window. The processed data can be channelled into files, databases, live dashboards, and more.